

TP1 – Packet Sniffing and Spoofing Lab

Duration: 6h

1. Lab Overview

Packet sniffing and spoofing are two important concepts in network security; they are two major threats in network communication. Being able to understand these two threats is essential for understanding security measures in networking. There are many packet sniffing and spoofing tools, such as Wireshark, Tcpdump, Netbox, Scapy, etc. Some of these tools are widely used by security experts, as well as by attackers. Being able to use these tools is important for students, but what is more important for students in a network security course is to understand how these tools work, i.e., how packet sniffing and spoofing are implemented in software.

The objective of this lab is two-fold: learning to use the tools and understanding the technologies underlying these tools. For the second objective, students will write simple sniffer and spoofing programs and gain an in-depth understanding of the technical aspects of these programs. This lab covers the following topics:

- How sniffing and spoofing work
- Packet sniffing using the pcap library and Scapy
- Packet spoofing using raw socket and Scapy
- Manipulating packets using Scapy

2. Learning Objectives

By the end of this lab, students will:

- Understand packet routing at a Virtual Machine and Container level;
- Capture packets and inspect their contents;
- Programmatically filter Network Layer traffic;
- Build small applications that spoof ICMP Echo Requests and ICMP Echo Responses.

3. Lab Requirements

- SEEDLabs Ubuntu [20.04 VM](#) ([22.04 for Mac](#))
- Internet access for downloading necessary packages.

4. Lab Setup

- Download the required files for the containers you will be using throughout the assignment:
 - Labsetup-TP1.zip
 - Labsetup-TP1-arm.zip (for Mac)

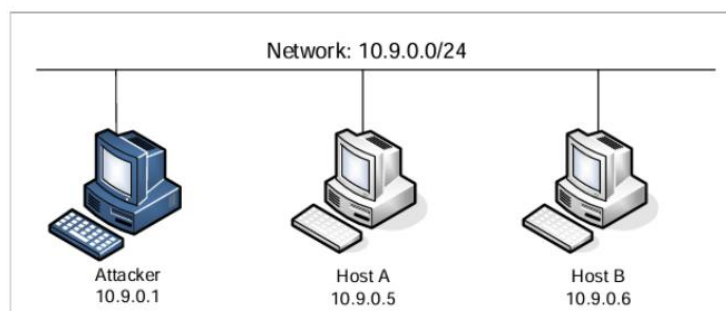


Figure 1: Lab environment setup

4.1 Container Setup and Commands

Download the Labsetup.zip file to your VM and use the docker-compose.yml file to set up the lab environment. In the following, we list some of the commonly used commands related to Docker and Compose.

```
$ docker-compose up -d # Start the containers
$ docker-compose down # Shut down the containers
```

All the containers will be running in the background. To run commands on a container, we often need to get a shell on that container.

We first need to use the "docker ps" command to find out the ID of the container and then use "docker exec -it <ID> <command>" to start a shell on that container.

5. Lab Task Set 1: Using Scapy to Sniff and Spoof Packets

Many tools can be used to do sniffing and spoofing, but most of them only provide fixed functionalities. Scapy is different: it can be used not only as a tool, but also as a building block to construct other sniffing and spoofing tools, i.e., we can integrate the Scapy functionalities into our own program. In this set of tasks, we will use Scapy for each task.

To use Scapy, we can write a Python program and then execute this program using Python. See the following example. We should run Python using the root privilege because it is required for spoofing packets.

```
#!/usr/bin/env python3

from scapy.all import *

a = IP()
a.show()
```

Make mycode.py executable:

```
chmod a+x mycode.py

./mycode.py
```

Run the code with python3:

```
python3 mycode.py
```

5.1. Sniffing Packets

Wireshark is the most popular sniffing tool, and it is easy to use. We will use it throughout the entire lab. However, it is difficult to use Wireshark as a building block to construct other tools. We will use Scapy for that purpose. The objective of this task is to learn how to use Scapy to do packet sniffing in Python programs. A sample code is provided in the following:

```
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    pkt.show()

pkt = sniff(iface='br-c94733e9f913', prn=print_pkt)
```

The code above will sniff the packets on the br-c94733e9f913 interface.

Task 1.1A: in the above program, for each captured packet, the callback function `print_pkt()` will be invoked; this function will print out some of the information about the packet. Run the program with the root privilege and demonstrate that you can indeed capture packets both in the VM as well as the attacker container. After that, run the program again, but without using the root privilege; describe and explain your observations.

Task 1.1B: usually, when we sniff packets, we are only interested in certain types of packets. We can do that by setting filters in sniffing. Scapy's filters use the BPF (Berkeley Packet Filter) syntax; you can find the BPF manual from the Internet. Please set the following filters and demonstrate your sniffer program again (each filter should be set separately):

- Capture only ICMP packets
- Capture any TCP packet that comes from a particular IP and with a destination port number 23.
- Capture packets that come from or go to a particular subnet. You can pick any subnet, such as 128.230.0.0/16; you should not pick the subnet that your VM is attached to.

Task 1.2 - Spoofing ICMP Packets: as a packet spoofing tool, Scapy allows us to set the fields of IP packets to arbitrary values. The objective of this task is to spoof IP packets with an arbitrary source IP address. We will spoof ICMP echo request packets and send them to another container. The following code shows an example of how to send an ICMP packet.

```
from scapy.all import *  
  
a = IP()  
a.dst = '10.0.2.3'  
b = ICMP()  
p = a/b  
send(p)
```

We can use `ls(a)` to see all the attribute names/values.

Spoof the source IP, show that the packet arrives at the destination and the response is sent to the spoofed IP.

Task 1.3 - Traceroute: the objective of this task is to use Scapy to estimate the distance, in terms of number of routers, between your VM and a selected destination. This is basically what is implemented by the `traceroute` tool. Write a program that achieves the same result (You will need to run this program in your native machine, why?).

Task 1.4 - Sniffing and then Spoofing: in this task, you will combine the sniffing and spoofing techniques to implement the following sniff-and-then-spoof program. You need two machines on the same LAN: the VM and the user container. From the user container, you ping an IP X. This will generate an ICMP echo request packet. If X is alive, the ping program will receive an echo reply and print out the response. Your sniff-and-then-spoof program runs on the VM, which monitors the LAN through packet sniffing. Whenever it sees an ICMP echo request, regardless of what the target IP address is, your program should immediately send out an echo reply using the packet spoofing technique. Therefore, regardless of whether machine X is alive or not, the ping program will always receive a reply, indicating that X is alive.

In your experiment, you should ping the following three IP addresses from the user container. Report your observation and explain the results.

```
ping 1.2.3.4  
ping 10.9.0.99  
ping 8.8.8.8
```

Hint: You will need to remember how the ARP protocol works to correctly explain your observation.

4. Lab Task Set 2: Writing Programs to Sniff and Spoof Packets

For this set of tasks, you should compile the C code inside the host VM and then run the code inside the container. You can use the `"docker cp"` command to copy a file from the host VM to a container. See the

following example (there is no need to type the docker ID in full and you may use the container's name as well):

```
$ docker ps
f4501a488a69  hostA-10.9.0.5
85058cbdee62  hostB-10.9.0.6
24cbc879e371  seed-attacker

docker cp a.out 24cbc879e371:/tmp
```

Task 2.1 - Writing Packet Sniffing Program: sniffer programs can be easily written using the pcap library. With pcap, the task of sniffers involves invoking a simple sequence of procedures in the pcap library. At the end of the sequence, packets will be put in buffer for further processing as soon as they are captured. All the details of packet capturing are handled by the pcap library. We include the following sample code:

```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>

void got_packet(u_char *args, const struct pcap_pkthdr *header, const u_char *packet){
    printf("Got a packet\n");
}

int main() {
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "icmp";
    bpf_u_int32 net;

    // Step 1: Open capture on desired interface
    handle = pcap_open_live("<interface>", BUFSIZ, 1, 1000, errbuf);

    // Step 2: Compile the filter
    pcap_compile(handle, &fp, filter_exp, 0, net);
    if (pcap_setfilter(handle, &fp) != 0) {
        pcap_perror(handle, "Error:"); exit(EXIT_FAILURE);
    }
    // Step 3: Capture packets
    pcap_loop(handle, -1, got_packet, NULL);

    pcap_close(handle);    //Close the handle
    return 0;
}

// Note: don't forget to add "-lpcap" to the compilation command.
// For example: gcc -o sniff sniff.c -lpcap
```

Task 2.1A Understanding How a Sniffer Works: in this task, you need to write a sniffer program to print out the source and destination IP addresses of each captured packet. You should provide screenshots as evidence to show that their sniffer program can run successfully and produce expected results. In addition, please answer the following questions:

- Question 1. Use your own words to describe the sequence of library calls that are essential for sniffer programs. This is meant to be a summary, not detailed explanation.
- Question 2. Why do you need the root privilege to run a sniffer program? Where does the program fail if it is executed without the root privilege?

Task 2.1B Writing Filters: write filter expressions for your sniffer program to capture each of the following. You can find online manuals for pcap filters. In your lab reports, you need to include screenshots to show the results after applying each of these filters.

- Capture the ICMP packets between two specific hosts.
- Capture the TCP packets with a destination port number in the range from 10 to 100.

Task 2.1C Sniffing Passwords: show how you can use your sniffer program to capture the password when somebody is using telnet on the network that you are monitoring. You may need to modify your sniffer code to print out the data part of a captured TCP packet (telnet uses TCP). It is acceptable if you print out the entire data part, and then manually mark where the password (or part of it) is.

Task 2.2 Spoofing: when a normal user sends out a packet, operating systems usually do not allow the user to set all the fields in the protocol headers (such as TCP, UDP, and IP headers). OSes will set most of the fields, while only allowing users to set a few fields, such as the destination IP address, the destination port number, etc. However, if users have the root privilege, they can set any arbitrary field in the packet headers. This is called packet spoofing, and it can be done through raw sockets.

Raw sockets give programmers the absolute control over the packet construction, allowing programmers to construct any arbitrary packet, including setting the header fields and the payload. Using raw sockets is quite straightforward; it involves four steps: (1) create a raw socket, (2) set socket option, (3) construct the packet, and (4) send out the packet through the raw socket. There are many online tutorials that can teach you how to use raw sockets in C programming. We show a simple skeleton of such a program:

```
#include <stdlib.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <stdio.h>

int main (void) {
    int sd;
    struct sockaddr_in sin;
    char buffer[1024];
    size_t ip_len;

    /* Create a raw socket with IP protocol. The IPPROTO_RAW parameter * tells the sytem that the IP header
    is already included, this prevents the OS from adding another IP header. */
    sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
    if(sd < 0) { perror("socket() error"); exit(-1); }

    /* This data structure is needed when sending the packets * using sockets. Normally, we need to fill out
    several fields, but for raw sockets, we only need to fill out * this one field */
    sin.sin_family = AF_INET;

    // Here you can construct the IP packet using buffer[]
    // - construct the IP header ...
    // - construct the TCP/UDP/ICMP header ...
    // - fill in the data part if needed ...
    // Note: you should pay attention to the network/host byte order.

    /* Send out the IP packet. ip_len is the actual size of the packet. */
    if(sendto(sd, buffer, ip_len, 0, (struct sockaddr *)&sin, sizeof(sin)) < 0) {
        perror("sendto() error");
        exit(-1);
    }
    return 0;
}
```

Task 2.2A Write a spoofing program: write your own packet spoofing program in C. You need to provide evidence to show that your program successfully sends out spoofed IP packets.

Task 2.2B Spoof an ICMP Echo Request: spoof an ICMP echo request packet on behalf of another machine (i.e., using another machine's IP address as its source IP address). This packet should be sent to a remote machine on the Internet (the machine must be alive). You should turn on your Wireshark, so that if your spoofing is successful, you can see the echo reply coming back from the remote machine.

- **Question 1.** Can you set the IP packet length field to an arbitrary value, regardless of how big the actual packet is?
- **Question 2.** Using raw socket programming, do you have to calculate the checksum for the IP header?
- **Question 3.** Why do you need the root privilege to run programs that use raw sockets? Where does the program fail if executed without the root privilege?

Task 2.3 Sniff and then Spoof: in this task, you will combine the sniffing and spoofing techniques to implement the following sniff-and-then-spoof program. You need two machines on the same LAN. From machine A, you ping an IP X. This will generate an ICMP echo request packet. If X is alive, the ping program will receive an echo reply and print out the response. Your sniff-and-then-spoof program runs on the attacker machine, which monitors the LAN through packet sniffing. Whenever it sees an ICMP echo request, regardless of what the target IP address is, your program should immediately send out an echo reply using the packet spoofing technique. Therefore, regardless of whether machine X is alive or not, the ping program will always receive a reply, indicating that X is alive. You need to write such a program in C and include screenshots in your report to show that your program works. Please also attach the code (with adequate amount of comments) in your report.

6. Report and submission

- A detailed report containing all of your answers will need to be delivered by the deadline for this assignment. Your responses should be clear and include enough information through text and/or screen captures that proves you understood the concept;
- Answers and/or code that was clearly written by AI will not be accepted. Those exercises will be marked with a 0;
- Indicate which question you are answering, and use the same order as stated in the PDF assignment;
- Write a brief conclusion with your learning experiences, as well as any point where you particularly struggled;
- Submission will be done through the BlackBoard Assignment hand-in. You should upload a single .zip file containing your report and code files. It should follow this naming scheme: NS-TPx-PL1y.zip, where x is the TP number and y is the working group ID. Your report and code files should follow the same scheme: NS-TPx-PL1y.pdf, NS-TPx-PL1y.c and NS-TPx-PL1y.py.